

Formalização do Algoritmo Genético para o Problema de Formação de Equipes de Software

Felipe Oliveira Miranda Cunha

1 Definição do Problema

Este trabalho investiga o Problema de Formação de Equipes de Software (*Software Team Formation Problem* – STFP). Uma instância do problema é definida por um conjunto de desenvolvedores disponíveis $D = \{d_1, d_2, \dots, d_N\}$, um projeto-alvo P com requisitos técnicos definidos por dimensão e prioridade, e o tamanho fixo k da equipe. O objetivo é selecionar uma equipe que apresente a maior aptidão para executar esse projeto.

A aptidão de uma equipe é determinada por dois componentes complementares:

- **Aptidão Técnica (AT)**: correspondência entre as competências técnicas dos desenvolvedores de uma equipe candidata e os requisitos técnicos do projeto-alvo. Esses requisitos são organizados em três dimensões: domínio de aplicação (DA), ecossistema de desenvolvimento (ED) e linguagem de programação (LP). Em cada dimensão, os requisitos são priorizados segundo o método MoSCoW, que os classifica como *Must-have*, *Should-have*, *Could-have* e *Won't-have*. Como os requisitos *Won't-have* não fazem parte do escopo da versão corrente do projeto, apenas as três primeiras categorias são consideradas na formulação [1].
- **Aptidão Colaborativa (AC)**: qualidade colaborativa potencial da equipe, estimada a partir da compatibilidade entre pares de desenvolvedores com base em histórico colaborativo em projetos anteriores trabalhados juntos.

Esses dois componentes são integrados em uma aptidão global (Aptidão da Equipe – AE), que representa o critério final de avaliação da equipe em relação ao projeto-alvo.

Uma equipe candidata é formada por um subconjunto de desenvolvedores. A equipe é considerada viável quando contém exatamente k desenvolvedores e cobre todos os requisitos *Must-have* definidos para o projeto-alvo. Assim, os requisitos *Must-have* são tratados como restrições rígidas do modelo. Entre todas as equipes viáveis, busca-se aquela que maximiza AE em relação ao projeto-alvo:

$$\max_{T \subseteq D, |T|=k, T \text{ cobre todos os } Must} AE(T, P),$$

em que D representa o conjunto de desenvolvedores disponíveis, T representa uma equipe candidata, P representa o projeto-alvo, k representa o tamanho fixo da equipe e $AE(T, P)$ representa a aptidão da equipe.

O método de solução adotado é um Algoritmo Genético (AG) que maximiza AE a partir de um avaliador surrogate calibrado. O AG busca no espaço combinatório $\mathcal{S} = \{T \subseteq D \mid |T| = k\}$ por meio de operadores estocásticos, preservando as restrições de viabilidade via política de penalidade rígida.

2 Formulação do Método de Solução

2.1 Entradas do Problema

Seja

$$D = \{d_1, d_2, \dots, d_N\}$$

o conjunto de desenvolvedores disponíveis, filtrado para conter apenas desenvolvedores presentes no grafo de colaboração. Cada desenvolvedor $d_i \in D$ possui competências técnicas associadas a três dimensões:

$$\Delta = \{\text{DA}, \text{ED}, \text{LP}\}.$$

Para cada dimensão $\delta \in \Delta$, seja U^δ o universo de competências possíveis naquela dimensão. As competências do desenvolvedor d_i na dimensão δ são representadas pelo parâmetro binário:

$$a_{iu}^\delta = \begin{cases} 1, & \text{se } d_i \text{ possui a competência } u \text{ na dimensão } \delta, \\ 0, & \text{caso contrário,} \end{cases}$$

para todo $d_i \in D$, $u \in U^\delta$ e $\delta \in \Delta$.

Adicionalmente, para cada par de desenvolvedores (d_i, d_j) com histórico de colaboração registrado, define-se a compatibilidade colaborativa bruta $PC_{ij} \in [0, 1]$, estimada a partir do histórico real de colaboração entre d_i e d_j . Por construção, $PC_{ij} = PC_{ji}$. O conjunto E reúne os pares com colaboração registrada. Cada valor PC_{ij} é convertido para um escore discretizado $\widehat{PC}_{ij}$, associado às faixas VL, L, M, H, VH, com centróides 0, 1, 0, 3, 0, 5, 0, 7 e 0, 9. Para pares fora de E , isto é, sem histórico de colaboração, adota-se o valor constante $\widehat{PC}_{\text{low}} = 0, 3$, correspondente à faixa *Low*, com base na regra elicitada com o especialista.

Para cada dimensão técnica $\delta \in \Delta$, o projeto P define requisitos organizados segundo a prioridade MoSCoW:

$$R_P^\delta = (M_P^\delta, S_P^\delta, C_P^\delta),$$

em que $M_P^\delta \subseteq U^\delta$ é o conjunto de requisitos *Must-have*, $S_P^\delta \subseteq U^\delta$ é o conjunto de requisitos *Should-have* e $C_P^\delta \subseteq U^\delta$ é o conjunto de requisitos *Could-have*, conjuntos disjuntos por construção.

Os coeficientes do escore dimensional $(b_1, b_2, w_{\text{covM}}, w_{\text{red2M}}, w_{\text{red3M}}, w_{\text{red4M}}, w_{\text{should}}, w_{\text{could}}, a_{\text{full}}, a_{\text{red2}})$, os pesos de dimensão $(w_{\text{DA}}, w_{\text{ED}}, w_{\text{LP}})$, os pesos da agregação global $(w_{\text{min}}, w_{\text{max}})$, o fator de compressão colaborativa $\gamma = 0,5$, o parâmetro *floor* $\eta \in [0, 1]$, o valor neutro $\mu_{\text{neutro}} \in [0, 1]$, os centróides colaborativos $\{0, 1, 0, 3, 0, 5, 0, 7, 0, 9\}$ e o valor de ausência de colaboração $\widehat{PC}_{\text{low}} = 0, 3$ são constantes do modelo. Os coeficientes e pesos foram obtidos na calibração do *surrogate*; γ foi incorporado no refinamento pós-validação para reduzir a amplitude operacional de AC, preservando sua ordem e seu ponto central. Esses parâmetros são descritos no capítulo do avaliador *surrogate*.

2.2 Representação do Cromossomo

No AG, a variável de decisão é uma equipe $T \subseteq D$ com $|T| = k$. Um cromossomo (indivíduo) representa uma equipe candidata como uma sequência ordenada de k identificadores inteiros de desenvolvedores distintos (gene):

$$\mathbf{c} = (c_1, c_2, \dots, c_k), \quad c_i \in D, \quad c_i \neq c_j \quad \forall i \neq j.$$

Cada gene c_i é o identificador numérico do desenvolvedor na base de dados; nenhum atributo adicional do desenvolvedor é armazenado no cromossomo. A ordem dos genes é relevante para o operador de cruzamento (a posição i do pai A combina com a posição i do pai B), mas a aptidão é calculada sobre o conjunto $\{c_1, \dots, c_k\}$ e é invariante em relação a permutações. Dois cromossomos são considerados idênticos se e somente se sua chave canônica

$$\kappa(\mathbf{c}) = \text{tuple}(\text{sorted}(c_1, \dots, c_k))$$

for igual.

O espaço de busca é:

$$\mathcal{S} = \{T \subseteq D \mid |T| = k\}, \quad |\mathcal{S}| = \binom{N}{k}.$$

Escore colaborativo médio como proxy escalar por desenvolvedor (gene). Para viabilizar o operador de Cruzamento por Combinação Convexa (CCC), descrito na Seção 2.6, associa-se a cada desenvolvedor um valor escalar $\phi : D \rightarrow [0, 1]$ calculado como a média dos pesos de colaboração de suas arestas no grafo G . Este valor serve exclusivamente como referência numérica para orientar a escolha de genes durante o cruzamento: não mede adequação técnica ao projeto-alvo (função exercida pelo componente AT do surrogate) nem entra no cálculo de aptidão da equipe. Trata-se de uma representação contínua de cada gene no cromossomo, necessária para que o CCC possa calcular alvos de mistura e comparar distâncias.

O valor é calculado como a média dos pesos das arestas do desenvolvedor no grafo de colaboração $G = (V, E, w)$:

$$\phi_{\text{raw}}(d_i) = \begin{cases} \frac{1}{|\mathcal{N}_i|} \sum_{d_j \in \mathcal{N}_i} w(d_i, d_j), & \text{se } |\mathcal{N}_i| > 0, \\ \phi_{\text{def}}, & \text{caso contrário,} \end{cases}$$

em que $\mathcal{N}_i = \{d_j \in D \mid (d_i, d_j) \in E\}$ é o conjunto de vizinhos de d_i no grafo e $w(d_i, d_j) \in [0, 1]$ é o peso da aresta de colaboração. O valor ϕ_{def} é atribuído a desenvolvedores sem nenhuma aresta no grafo e entra na normalização min-max junto com os demais. O escore é então normalizado sobre D :

$$\phi(d_i) = \frac{\phi_{\text{raw}}(d_i) - \phi_{\min}}{\phi_{\max} - \phi_{\min}},$$

com $\phi_{\min} = \min_{d \in D} \phi_{\text{raw}}(d)$ e $\phi_{\max} = \max_{d \in D} \phi_{\text{raw}}(d)$.

Exemplo do CCC na posição i do cromossomo. Considere dois pais com os seguintes genes e escores na posição 1: $\phi(\text{Dev_10}) = 0,80$ (Pai A) e $\phi(\text{Dev_07}) = 0,30$ (Pai B). Com $\alpha_1 = 0,70$ sorteado entre $[0, 1]$ para essa posição, o alvo é:

$$\phi_c^{(1)} = 0,70 \times 0,80 + 0,30 \times 0,30 = 0,65.$$

As distâncias em relação ao alvo $\phi_c = 0,65$ são $|\phi_c - \phi_A| = 0,15$ e $|\phi_c - \phi_B| = 0,35$, de modo que o filho herda Dev_10 nessa posição por ser o de menor distância. A Figura 1 ilustra esse mecanismo na reta numérica: o alvo cai sempre dentro do intervalo $[\phi_B, \phi_A]$, propriedade que define a combinação convexa e impede que o operador produza valores fora da faixa dos progenitores.

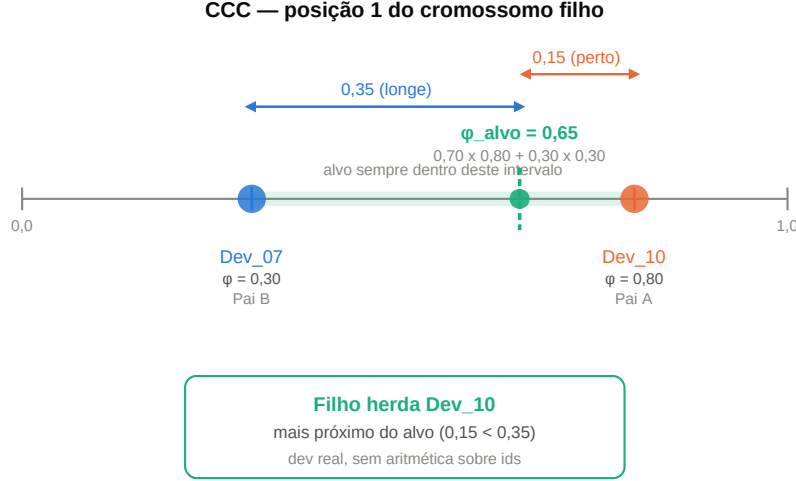


Figura 1: Ilustração do CCC na posição 1 do cromossomo filho. O alvo $\phi_c = 0,65$, calculado com $\alpha = 0,70$, cai dentro do intervalo $[0,30, 0,80]$. O filho herda Dev_10 por estar mais próximo do alvo ($0,15 < 0,35$).

2.3 Inicialização da População

A população inicial $\mathcal{P}_0$ é composta por λ cromossomos gerados por amostragem aleatória uniforme sem reposição:

$$\mathbf{c}_0^{(j)} = \text{sample_without_replacement}(D, k), \quad j = 1, 2, \dots, \lambda.$$

Não são impostas restrições de viabilidade na inicialização: equipes que não cobrem requisitos *Must-have* são geradas normalmente e penalizadas na avaliação de aptidão. Uma inicialização guiada (*warm start*) que garanta cobertura dos requisitos *Must-have* desde a população inicial constitui uma extensão natural desta abordagem e é deixada como trabalho futuro. A reprodutibilidade é garantida pela fixação de uma semente aleatória σ no início de cada execução do AG, tornando toda a sequência de operações estocásticas determinística para um dado valor de σ .

2.4 Função de Aptidão

A aptidão de um cromossomo $\mathbf{c}$ é calculada pelo avaliador surrogate calibrado, denotado $\mathcal{F}$. O avaliador recebe o conjunto de desenvolvedores da equipe e o projeto-alvo P e retorna a aptidão global estimada $\widehat{AE}(\mathbf{c}, P) \in [0, 1]$, conforme o modelo surrogate descrito no capítulo correspondente.

Avaliador *surrogate* refinado. Para explicitar a função efetivamente otimizada pelo AG, seja $\text{cov}_\pi^\delta \in [0, 1]$ a proporção de requisitos cobertos na prioridade $\pi \in \{M, S, C\}$ da dimensão δ . Para os requisitos obrigatórios, $\text{red}_{r,M}^\delta$ representa a proporção coberta por pelo menos r integrantes. No caso em que duas ou mais prioridades estão ativas, o

argumento do escore dimensional é:

$$v_C^\delta = b_1 + w_{\text{covM}} \text{cov}_M^\delta + w_{\text{red2M}} \text{red}_{2,M}^\delta + w_{\text{red3M}} \text{red}_{3,M}^\delta + w_{\text{red4M}} \text{red}_{4,M}^\delta + w_{\text{should}} (\text{cov}_M^\delta \text{cov}_S^\delta) + w_{\text{could}} (\text{cov}_M^\delta \text{cov}_C^\delta).$$

Os termos de *Should-have* e *Could-have* são independentes: aumentar cov_S^δ não reduz a contribuição de cov_C^δ . Quando não há requisitos *Must-have* na dimensão, adota-se $\text{cov}_M^\delta = 1$ como valor neutro. Os pesos usados são $b_1 = 0$, $w_{\text{covM}} = 0$, $w_{\text{red2M}} = 0,4120879121$, $w_{\text{red3M}} = 0,2582417582$, $w_{\text{red4M}} = 0$ e $w_{\text{should}} = w_{\text{could}} = 0,2307692308$. O termo $\text{red}_{4,M}^\delta$ é preservado para rastreabilidade, mas não altera o escore porque seu peso é zero. Após a limitação ao intervalo $[0, 1]$ e o tratamento de cobertura obrigatória definido no modelo dimensional, obtém-se DIM^δ .

A aptidão técnica é então:

$$\widehat{AT}(\mathbf{c}, P) = \frac{3 \text{DIM}^{\text{DA}} + \text{DIM}^{\text{ED}} + 5 \text{DIM}^{\text{LP}}}{9}.$$

Se $p_\ell(\mathbf{c})$ é a proporção dos pares da equipe na faixa colaborativa $\ell \in \{\text{VL}, \text{L}, \text{M}, \text{H}, \text{VH}\}$ e $c_\ell \in \{0,1,0,3,0,5,0,7,0,9\}$ é o respectivo centróide, o escore colaborativo original é:

$$AC_0(\mathbf{c}) = \sum_{\ell} p_\ell(\mathbf{c}) c_\ell.$$

O refinamento comprime esse escore em torno de 0,5:

$$\widehat{AC}(\mathbf{c}) = 0,5 + \gamma (AC_0(\mathbf{c}) - 0,5), \quad \gamma = 0,5,$$

ou, de forma equivalente, $\widehat{AC}(\mathbf{c}) = 0,25 + 0,5 AC_0(\mathbf{c})$. Por fim, a aptidão global utilizada pelo AG permanece a função *mixminmax* calibrada:

$$\widehat{AE}(\mathbf{c}, P) = \frac{5 \min(\widehat{AT}, \widehat{AC}) + \max(\widehat{AT}, \widehat{AC})}{6}.$$

Política de viabilidade rígida (*Hard-Must*). Para cada dimensão $\delta \in \Delta$, define-se o indicador de cobertura de requisitos *Must-have*:

$$\mu^\delta(\mathbf{c}, P) = \begin{cases} 1, & \text{se } \forall u \in M_P^\delta : \exists c_i \in \mathbf{c} \text{ tal que } a_{iu}^\delta = 1, \\ 0, & \text{caso contrário.} \end{cases}$$

Quando uma dimensão δ não possui requisitos *Must-have* definidos no projeto ($M_P^\delta = \emptyset$), adota-se $\mu^\delta(\mathbf{c}, P) = 1$ por convenção, ou seja, a dimensão não restringe a viabilidade da equipe. A aptidão efetiva adotada pelo AG é:

$$f(\mathbf{c}) = \begin{cases} \widehat{AE}(\mathbf{c}, P), & \text{se } \mu^{\text{DA}}(\mathbf{c}, P) = \mu^{\text{ED}}(\mathbf{c}, P) = \mu^{\text{LP}}(\mathbf{c}, P) = 1, \\ f_{\text{inv}}, & \text{caso contrário,} \end{cases}$$

em que $f_{\text{inv}} = -1,0$ é a penalidade de inviabilidade, garantindo que qualquer equipe viável supere qualquer equipe inviável, pois $\widehat{AE} \in [0, 1]$.

Cache de aptidão. Para cada chave canônica $\kappa(\mathbf{c})$, o resultado bruto do surrogate é armazenado em um cache $\mathcal{C} : \kappa \mapsto \mathcal{F}(\mathbf{c}, P)$. Ao avaliar um cromossomo, verifica-se primeiro se $\kappa(\mathbf{c}) \in \mathcal{C}$; em caso positivo, o resultado é recuperado sem nova chamada ao surrogate e a política *Hard-Must* é reaplicada sobre o resultado em cache.

2.5 Operador de Seleção

A seleção de pais é realizada por **seleção por truncamento**, operador clássico da computação evolucionária [8].

A cada geração, a população $\mathcal{P}_g$ é ordenada em ordem decrescente de aptidão e define-se o **pool de seleção** como os κ_{pool} melhores indivíduos:

$$\kappa_{\text{pool}} = \max\left(2, \left\lfloor \frac{\lambda}{2} \right\rfloor\right).$$

Com $\lambda = 150$, tem-se $\kappa_{\text{pool}} = 75$, ou seja, a metade superior da população. Dois pais distintos são sorteados uniformemente sem reposição dentro do pool:

$$(\mathbf{p}_1, \mathbf{p}_2) \sim \text{Uniform_sample}\left(\mathcal{P}_g^{\text{top}[\kappa_{\text{pool}}]}, 2, \text{sem reposição}\right).$$

Nota: neste trabalho, λ denota o tamanho total da população, seguindo a convenção dos Algoritmos Genéticos clássicos [6], e não a convenção (μ, λ) das Estratégias Evolucionárias.

Os indivíduos preservados pelo elitismo ($\mathcal{E}_g$) integram a população $\mathcal{P}_{g-1}$ e, por estarem entre os melhores, fazem parte do pool de seleção. Portanto, os elites também podem ser sorteados como pais, além de serem copiados diretamente para a nova geração.

Justificativa da escolha. Análises teóricas sobre distribuição de aptidão demonstram que truncamento a 50% e torneio binário apresentam intensidades seletivas equivalentes em expectativa, diferindo apenas na distribuição interna da pressão entre os sobreviventes [3, 6]. Bäck [2] posiciona a seleção (μ, λ) no mesmo eixo de pressão seletiva que torneio e seleção proporcional, confirmando a equivalência quando a intensidade é igualada. Em SBSE, Conrad et al. [4] compararam explicitamente truncamento, torneio e roleta em um problema de priorização de suites de teste por busca e constataram que o truncamento superou consistentemente as demais modalidades em efetividade. A configuração completa foi submetida a calibração sistemática e validada empiricamente contra a solução ótima obtida por Programação Linear Inteira e contra o ranqueamento de um especialista humano em ambiente industrial real.

2.6 Operador de Cruzamento

O operador de cruzamento adotado é uma adaptação do Cruzamento por Combinação Convexa (CCC, *Convex Combination Crossover*) [7, 5] para representações discretas de conjuntos, ativado com probabilidade $p_c \in [0, 1]$. Quando não ativado, os filhos são cópias diretas dos pais.

Justificativa. O operador de Cruzamento por Combinação Convexa foi originalmente proposto por [7] para otimização de parâmetros contínuos de um algoritmo de colônia de formigas aplicado ao problema do caixeiro viajante. Naquele contexto, os genes são vetores

de números reais e a combinação convexa é aplicada diretamente sobre eles, gerando filhos com valores intermediários entre os dois progenitores:

$$Z_1 = \theta c_1 + (1 - \theta)c_2.$$

No STFP, os genes são identificadores inteiros de desenvolvedores e não admitem interpolação aritmética direta: não existe um desenvolvedor que seja a média ponderada de outros dois. A adaptação proposta em trabalho anterior [5] aplica a combinação convexa sobre os escores colaborativos ϕ , gerando um alvo $\phi_c^{(i)} \in [\phi_B, \phi_A]$ que orienta a seleção do gene parental mais próximo em cada posição. O princípio de convexidade é preservado: o alvo nunca extrapola o intervalo entre os escores dos dois progenitores. Diferentemente de operadores posicionais como o cruzamento de dois pontos (*two-point crossover*) ou o cruzamento baseado em posição (*position-based crossover*), o operador não opera sobre posições fixas de corte, mas realiza uma interpolação estocástica dos escores gene a gene, orientando a herança em direção à combinação convexa dos escores colaborativos dos progenitores. A unicidade dos genes em cada filho é garantida pelo Passo 5, que substitui duplicatas pelo desenvolvedor disponível com ϕ mais próximo do alvo. A validação empírica demonstrou que o operador superou estatisticamente o *Partially Mapped Crossover* (PMX) e o cruzamento de um ponto em relação ao fitness médio, fitness máximo e diversidade [5].

Definição formal. Dados os pais $\mathbf{p}_1 = (p_{1,1}, \dots, p_{1,k})$ e $\mathbf{p}_2 = (p_{2,1}, \dots, p_{2,k})$, o operador produz dois filhos $\mathbf{f}_1$ e $\mathbf{f}_2$ de forma independente por meio do procedimento GenFilho($\mathbf{p}_1, \mathbf{p}_2$).

Para cada posição $i \in \{1, \dots, k\}$:

Passo 1. Obtenha os escores colaborativos médios (proxy) dos genes parentais:

$$\phi_A = \phi(p_{1,i}), \quad \phi_B = \phi(p_{2,i}).$$

Passo 2. Gere um coeficiente de mistura:

$$\alpha_i \sim \text{Uniform}(0, 1).$$

Passo 3. Calcule o escore convexo-alvo:

$$\phi_c^{(i)} = \alpha_i \cdot \phi_A + (1 - \alpha_i) \cdot \phi_B.$$

Passo 4. Selecione o gene do pai mais próximo de $\phi_c^{(i)}$:

$$g^{(i)} = \begin{cases} p_{1,i}, & \text{se } |\phi_c^{(i)} - \phi_A| \leq |\phi_c^{(i)} - \phi_B|, \\ p_{2,i}, & \text{caso contrário.} \end{cases}$$

Em caso de empate exato ($|\phi_c^{(i)} - \phi_A| = |\phi_c^{(i)} - \phi_B|$), o gene do pai $\mathbf{p}_1$ é selecionado como consequência do operador de comparação $\leq$ utilizado na implementação. Como os dois pais são sorteados uniformemente do pool de seleção, esse comportamento não introduz viés sistemático em favor de nenhum dos progenitores ao longo das gerações.

Passo 5. Se $g^{(i)}$ já estiver no filho em construção, defina o conjunto de candidatos disponíveis:

$$\mathcal{A}_i = D_{\text{valid}} \setminus \{f_1, f_2, \dots, f_{i-1}\},$$

em que $D_{\text{valid}} \subseteq D$ é o subconjunto de desenvolvedores presentes no grafo de colaboração e $\{f_1, \dots, f_{i-1}\}$ são os genes já atribuídos ao filho. Substitua $g^{(i)}$ pelo candidato mais próximo do alvo:

$$g^{(i)} \leftarrow \arg \min_{d \in \mathcal{A}_i} |\phi(d) - \phi_c^{(i)}|.$$

Em caso de empate, qualquer minimizador é aceito; se $\mathcal{A}_i = \emptyset$, seleciona-se um gene aleatório de $D \setminus \{f_1, \dots, f_{i-1}\}$.

Passo 6. Atribua $f_i \leftarrow g^{(i)}$.

O procedimento completo é:

$$\mathbf{f}_1 = \text{GenFilho}(\mathbf{p}_1, \mathbf{p}_2), \quad \mathbf{f}_2 = \text{GenFilho}(\mathbf{p}_1, \mathbf{p}_2).$$

Os dois filhos são independentes entre si porque os coeficientes α_i são amostrados novamente para cada filho.

Por construção, o procedimento GenFilho garante que cada filho é uma sequência de k genes distintos pertencentes a D_{valid} , preservando a validade estrutural do cromossomo independentemente da cobertura de requisitos *Must-have*, que é verificada posteriormente pela função de aptidão.

Uma variante possível consiste em orientar o sorteio de α_i para valores acima de 0,5 quando o pai $\mathbf{p}_1$ apresentar score colaborativo médio superior ao pai $\mathbf{p}_2$, aumentando a probabilidade de herdar genes do progenitor colaborativamente mais experiente.

2.7 Operador de Mutação

A mutação é aplicada por cromossomo com probabilidade p_m . Dado $\mathbf{c} = (c_1, \dots, c_k)$:

Passo 1. Gere $u \sim \text{Uniform}(0, 1)$.

Passo 2. Se $u < p_m$, sorteie um índice $i^* \sim \text{Uniform}\{1, \dots, k\}$.

Passo 3. Substitua c_{i^*} por um desenvolvedor sorteado uniformemente do pool de mutação:

$$c_{i^*} \leftarrow \text{sample_uniform}(D_{\text{valid}} \setminus \{c_1, \dots, c_k\}).$$

A restrição de unicidade dos genes é preservada por construção. Formalmente:

$$P(\text{cromossomo é mutado}) = p_m, \quad P(i^* = i \mid \text{mutação ocorre}) = \frac{1}{k}.$$

2.8 Elitismo

O AG preserva os ε melhores indivíduos únicos entre gerações consecutivas. O conjunto de elites é construído iterativamente a partir da população ordenada:

$$\mathcal{E}_g = \{\mathbf{c}^{(j)} \in \mathcal{P}_g \mid \kappa(\mathbf{c}^{(j)}) \text{ é distinto dos anteriores em } \mathcal{E}_g, |\mathcal{E}_g| \leq \varepsilon\}.$$

Os indivíduos em $\mathcal{E}_g$ são copiados diretamente para $\mathcal{P}_{g+1}$ sem a aplicação de cruzamento ou mutação. Na configuração calibrada, $\varepsilon = 3$.

2.9 Controle de Unicidade da População

O AG impõe que todos os cromossomos de uma mesma geração sejam únicos segundo κ . A verificação é feita por consulta a um conjunto $\mathcal{K}_{\text{seen}}$ de chaves canônicas já presentes na nova população, operação de custo $O(1)$ por consulta. Ao gerar um filho cuja chave já está em $\mathcal{K}_{\text{seen}}$, aplica-se uma sequência de mutações forçadas:

$$\mathbf{f} \leftarrow \text{ForceMutate}(\mathbf{f}, \mathcal{K}_{\text{seen}}, m_{\text{max}}),$$

em que $\mathcal{K}_{\text{seen}}$ é o conjunto de chaves já presentes na nova população e $m_{\text{max}} = 30$ é o número máximo de tentativas de mutação elementar. O limite global de tentativas de geração de filhos é:

$$\text{MAX_TRIES} = 40 \cdot (\lambda - |\mathcal{E}_g|),$$

em que $|\mathcal{E}_g| \leq \varepsilon$ é o número efetivo de elites únicos preservados.

2.10 Critérios de Parada

O AG termina quando qualquer um dos dois critérios a seguir é satisfeito.

Critério 1 – Número máximo de gerações.

$$g = G_{\text{max}}.$$

Critério 2 – Estagnação do melhor indivíduo. Seja $f^*(g) = \max_{\mathbf{c} \in \mathcal{P}_g} f(\mathbf{c})$. O contador de estagnação s_g é definido por:

$$s_g = \begin{cases} 0, & \text{se } f^*(g) > f_{\text{ever}}^* + \delta_{\text{min}} \text{ e } f^*(g) > f_{\text{inv}}, \\ 0, & \text{se } f^*(g) \leq f_{\text{inv}} \text{ (nenhuma equipe viável ainda),} \\ s_{g-1} + 1, & \text{caso contrário,} \end{cases}$$

em que f_{ever}^* é o melhor fitness global até $g - 1$ e $\delta_{\text{min}} = 10^{-4}$. O AG para por estagnação quando:

$$s_g \geq S_{\text{max}}.$$

Ao término, o AG retorna o melhor indivíduo global em todas as gerações:

$$\mathbf{c}^* = \arg \max_{g \in \{0, \dots, G\}} f^*(g).$$

2.11 Seleção de Equipes Diversas para Apresentação

Após o término, o sistema apresenta ao gestor não apenas a melhor equipe encontrada, mas até quatro alternativas com diversidade mínima entre si. O objetivo é oferecer opções com composições distintas, permitindo que o gestor considere fatores não modelados, como disponibilidade ou preferências organizacionais. A diversidade entre duas equipes T_a e T_b é medida pela distância de Jaccard:

$$d_J(T_a, T_b) = 1 - \frac{|T_a \cap T_b|}{|T_a \cup T_b|}.$$

As equipes são selecionadas iterativamente em ordem decrescente de aptidão, com limiar de distância mínima relaxado progressivamente em $\{0, 5, 0, 4, 0, 3, 0, 2, 0, 1\}$ até quatro equipes serem selecionadas. A equipe $\mathbf{c}^*$ é sempre incluída como primeira selecionada.

2.12 Tabela de Hiperparâmetros

Tabela 1: Hiperparâmetros do AG (configuração calibrada pelo racing adaptativo).

Símbolo	Nome	Valor	Justificativa
λ	Tamanho da população	150	Selecionado pelo procedimento de racing adaptativo; supera a linha de base ($\lambda = 100$) em todos os estágios de calibração.
$G_{\max}$	Máximo de gerações	200	Limite superior; na prática interrompido pelo critério de estagnação.
$S_{\max}$	Gerações sem melhora	20	Critério de tolerância mais amplo selecionado pelo racing; reduz paradas prematuras.
p_c	Taxa de cruzamento	1,0	parâmetro sensível; maximiza recombinação a cada geração.
p_m	Taxa de mutação	0,005	Selecionado pelo racing; inferior à linha de base ($p_m = 0,05$), reduzindo perturbação e favorecendo convergência.
ε	Elitismo	3	Selecionado pelo racing; superior à linha de base ($\varepsilon = 1$) em qualidade e estabilidade.
κ_{pool}	Pool de seleção	$\max(2, \lfloor 150/2 \rfloor)$ 75	Truncamento a 50%; pressão seletiva equivalente ao torneio binário [3].
$\delta_{\min}$	Limiar de melhoria	10^{-4}	Distingue melhorias reais de flutuações numéricas.
f_{inv}	Penalidade de inviabilidade	-1,0	Garante que qualquer solução viável supere qualquer inviável.
ϕ_{def}	Proxy padrão (sem aresta no grafo)	0,5	Atribuído a desenvolvedores sem arestas no grafo; entra na normalização min-max junto com os demais.
$m_{\max}$	Tentativas de mutação forçada	30	Limita o custo do reparo de duplicatas.
σ	Semente aleatória	123	Fixada para reprodutibilidade dos experimentos.

A Tabela 1 apresenta os valores calibrados adotados. Os intervalos de busca completos utilizados no procedimento de racing adaptativo estão descritos no capítulo de calibração da tese. A Tabela 2 reproduz um extrato dos principais intervalos para referência imediata.

Tabela 2: Extrato dos intervalos de busca usados na calibração (valores completos na Tabela de calibração da tese).

Símbolo	Nome	Intervalo de busca
λ	Tamanho da população	$\{30, 50, 80, 100, 150, 200\}$
$G_{\max}$	Máximo de gerações	$\{30, 50, 100, 150, 200\}$
$S_{\max}$	Gerações sem melhora	$\{3, 5, 10, 15, 20\}$
p_c	Taxa de cruzamento	$\{0,60, 0,80, 0,90, 1,00\}$
p_m	Taxa de mutação	$\{0,005, 0,010, 0,020, 0,050, 0,080, 0,100\}$
ε	Elitismo	$\{0, 1, 2, 3, 5\}$

2.13 Pseudocódigo do Algoritmo

Algorithm 1 Algoritmo Genético para Formação de Equipes (AG-STFP)

Require: $D, P, k, \lambda, G_{\max}, S_{\max}, p_c, p_m, \varepsilon, \delta_{\min}, \sigma$

Ensure: Melhor equipe $\mathbf{c}^*$ e aptidão f^*

```

1: Fixar semente  $\sigma$ ; limpar cache  $\mathcal{C} \leftarrow \emptyset$ 
2: Calcular  $\phi(d_i)$  para todo  $d_i \in D$ 

3: // Inicialização
4: for  $j = 1$  até  $\lambda$  do
5:    $\mathbf{c}_0^{(j)} \leftarrow \text{sample\_without\_replacement}(D, k)$ 
6: end for
7:  $\mathcal{P}_0 \leftarrow \{\mathbf{c}_0^{(1)}, \dots, \mathbf{c}_0^{(\lambda)}\}$ 
8: Avaliar  $f(\mathbf{c})$  para todo  $\mathbf{c} \in \mathcal{P}_0$  ▷ Usa cache  $\mathcal{C}$ 
9:  $f_{\text{ever}}^* \leftarrow \max f(\mathbf{c})$ ;  $\mathbf{c}^* \leftarrow \arg \max f(\mathbf{c})$ ;  $s \leftarrow 0$ 

10: // Loop evolutivo
11: for  $g = 1$  até  $G_{\max}$  do
12:   // Elitismo
13:    $\mathcal{E} \leftarrow \text{Top}_{\varepsilon}^{\text{unique}}(\mathcal{P}_{g-1})$ 
14:    $\mathcal{K} \leftarrow \{\kappa(\mathbf{c}) \mid \mathbf{c} \in \mathcal{E}\}$ ;  $\mathcal{Q} \leftarrow \mathcal{E}$ 

15:   // Geração de filhos
16:   while  $|\mathcal{Q}| < \lambda$  do
17:      $(\mathbf{p}_1, \mathbf{p}_2) \leftarrow \text{sample}(\text{Top}_{\kappa_{\text{pool}}}(\mathcal{P}_{g-1}), 2)$  ▷ Truncamento a 50%
18:     if  $u \sim \text{Unif}(0, 1) < p_c$  then
19:        $(\mathbf{f}_1, \mathbf{f}_2) \leftarrow \text{CCC}(\mathbf{p}_1, \mathbf{p}_2)$  ▷ Cruzamento por Combinação Convexa
20:     else
21:        $\mathbf{f}_1 \leftarrow \mathbf{p}_1$ ;  $\mathbf{f}_2 \leftarrow \mathbf{p}_2$ 
22:     end if
23:     for  $\mathbf{f} \in \{\mathbf{f}_1, \mathbf{f}_2\}$  do
24:        $\mathbf{f} \leftarrow \text{Mutacao}(\mathbf{f}, p_m)$ 
25:       if  $\kappa(\mathbf{f}) \in \mathcal{K}$  then
26:          $\mathbf{f} \leftarrow \text{ForceMutate}(\mathbf{f}, \mathcal{K}, m_{\max})$ 
27:       end if
28:       if  $\kappa(\mathbf{f}) \notin \mathcal{K}$  e  $|\mathcal{Q}| < \lambda$  then
29:          $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{\mathbf{f}\}$ ;  $\mathcal{K} \leftarrow \mathcal{K} \cup \{\kappa(\mathbf{f})\}$ 
30:       end if
31:     end for
32:   end while
33:    $\mathcal{P}_g \leftarrow \mathcal{Q}$ 
34:   Avaliar  $f(\mathbf{c})$  para todo  $\mathbf{c} \in \mathcal{P}_g$  ▷ Usa cache  $\mathcal{C}$ ; surrogate chamado apenas se  $\kappa(\mathbf{c}) \notin \mathcal{C}$ 

35:   // Atualização e critério de parada
36:    $f_g^* \leftarrow \max_{\mathbf{c} \in \mathcal{P}_g} f(\mathbf{c})$ 
37:   if  $f_g^* \leq f_{\text{inv}}$  then
38:      $s \leftarrow 0$ 
39:   else if  $f_g^* > f_{\text{ever}}^* + \delta_{\min}$  then
40:      $f_{\text{ever}}^* \leftarrow f_g^*$ ;  $\mathbf{c}^* \leftarrow \arg \max f(\mathbf{c})$ ;  $s \leftarrow 0$ 
41:   else
42:      $s \leftarrow s + 1$ 
43:   end if
44:   if  $s \geq S_{\max}$  then
45:     break
46:   end if

47: end for
48: return  $\mathbf{c}^*, f_{\text{ever}}^*, \text{SelectDiverse}(\mathcal{P}_g, \mathbf{c}^*, d_J)$ 

```

Referências

- [1] Sultan Alshehri and Luigi Benedicenti. Ranking approach for the user story prioritization methods. *Journal of Communication and Computer*, 10(12):1465–1474, 2013.
- [2] T. Back. Selective pressure in evolutionary algorithms: a characterization of selection mechanisms. In *Proceedings of the First IEEE Conference on Evolutionary Computation. IEEE World Congress on Computational Intelligence*, pages 57–62 vol.1, 1994.
- [3] Tobias Blicke and Lothar Thiele. A comparison of selection schemes used in evolutionary algorithms. *Evolutionary Computation*, 4(4):361–394, 1996.
- [4] Alexander P. Conrad, Robert S. Roos, and Gregory M. Kapfhammer. Empirically studying the role of selection operators during search-based test suite prioritization. In *Proceedings of the 12th Annual Conference on Genetic and Evolutionary Computation, GECCO '10*, page 1373–1380, New York, NY, USA, 2010. Association for Computing Machinery.
- [5] Felipe Cunha, Mirko Perkusich, Danylo Albuquerque, Kyller Gorgônio, Hyggo Almeida, and Angelo Perkusich. A genetic algorithm with convex combination crossover for software team formation: Integrating technical and collaboration skills. In *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing, SAC '25*, page 146–153, New York, NY, USA, 2025. Association for Computing Machinery.
- [6] David E. Goldberg and Kalyanmoy Deb. A comparative analysis of selection schemes used in genetic algorithms. volume 1 of *Foundations of Genetic Algorithms*, pages 69–93. Elsevier, 1991.
- [7] Wu Liu-ai and Fan Wen-Qing. A parameter model of genetic algorithm regulating ant colony algorithm. In *2012 IEEE Ninth International Conference on e-Business Engineering*, pages 50–54, 2012.
- [8] Heinz Mühlenbein and Dirk Schlierkamp-Voosen. Predictive models for the breeder genetic algorithm i. continuous parameter optimization. *Evolutionary Computation*, 1(1):25–49, 03 1993.